OOP prinsipləri

OOP nin 4 əsas prinsipi var:

- Abstraction
- Encapsulation
- Inheritance
- Polymorphism

Abstraction nədir?

- C#-da **abstraction** (abstraksiya) obyekt yönümlü proqramlaşdırmanın (OOP) əsas prinsiplərindən biridir və **detalları gizlədib, yalnız lazım olan hissəni göstərmək** deməkdir.

- Sadə dildə desək istifadəçiyə funksionallığın nə etdiyini göstəririk,necə etdiyini isə gizlədirik.

- Misal olaraq:

Telefonun kamera düyməsini basanda şəkil çəkilir.

- İstifadəçi üçün məlum olan:”Düyməni bas,şəkil çəkilsin.”
- Gizlədilən hissə : Kamera sensorunun necə işə düşməsi,işığın necə rəqəmsal siqnala çevrilməsi,faylın yaddaşa yazılması vəs.

csharp

```csharp
using System;

// Abstrakt sinif - istifadəçiyə yalnız "nə edir" göstərilir
abstract class Camera
{
    // İstifadəçiyə görünən metod imzası
    public abstract void TakePhoto();
}

// İmplementasiya sinfi - "necə edir" hissəsi burada gizlidir
class DigitalCamera : Camera
{
    public override void TakePhoto()
    {
        Console.WriteLine("📷 Kamera sensoru işə düşdü...");
        Console.WriteLine("💡 İşıq toplanır...");
        Console.WriteLine("🖼️ Şəkil yaddaşa yazıldı.");
    }
}

class Program
{
    static void Main()
    {
        // İstifadəçi abstrakt sinifi interfeys kimi görür
        Camera camera = new DigitalCamera();

        // İstifadəçi üçün sadə əməliyyat:
        camera.TakePhoto(); // "nə edir" hissəsi
    }
}
```

Niyə abstraction istifadə olunur?

- **Kodun mürəkkəbliyini azaldır** — istifadəçi implementasiya detallarını görmür.
- **Təhlükəsizlik** — vacib olmayan və ya həssas detallar gizlədilir.
- **Yenidən istifadə** — ümumi interfeyslərlə müxtəlif implementasiyalar yaradıla bilər.
- **Daha rahat dəyişiklik** — daxili kod dəyişsə də, istifadəçi interfeysi dəyişmir.

C#-da abstraction necə edilir?Abstract class nədir?

- Abstraksiya əsasən **abstract class** və **interface** vasitəsilə həyata keçirilir.

- abstract açar sözü ilə yaradılan class-dır.Onun birbaşa obyektini yaratmaq olmaz.(new ilə yeni instance)
- # Yalnız ondan miras alan class dan obyekt yaradıla bilər.
- Bu class həm tam implementasiya olunmuş metodlar,həm də abstract metodlar(body siz metodlar) saxlaya bilər.

Xüsusiyyətləri:

Instance yaratmaq olmaz:

Shape shape = new Shape(); // ✕ error

Abstract metodlar override edilməlidir:

public abstract double GetArea();

Normal (tam) metodlar da yaza bilərsən:

public void Display() => Console.WriteLine("This is a shape");

Field ,property,constructor saxlaya bilər:
Interface lərdən fərqli olaraq abstract class-lar state saxlayır.

Bir class yalnız bir abstract class-dan miras ala bilər:

Bu, C#-ın single inheritance qaydasına görədir.

```
abstract class Shape
{
    public abstract double GetArea(); // implementasiya yoxdur
    public void Display() => Console.WriteLine("This is a shape."); // var
}

class Circle : Shape
{
    private double radius;
    public Circle(double radius) => this.radius = radius;
    public override double GetArea() => Math.PI * radius * radius;
}
```

"State" nə deməkdir?

- **State** dedikdə obyektin **daxili vəziyyəti** nəzərdə tutulur.
  Bu vəziyyət obyektin **field** və **property**-lərində saxlanır.

- class User
- {
- public string Name { get; set; } // property → state saxlayır
- private int age; // field → state saxlayır
- }

Interface nədir?

- Interface sadəcə **metod, property, event və indexer-lərin imzalarını** saxlayır.
  C# 8-dən etibarən **default implementasiya** da mümkündür, amma yenə də sahə (field) saxlamaz.

İmplementasiya nədir?

•**Deklarasiya (təsvir)** → metodun *nə* edəcəyini deyir (imza).
•**Implementasiya** → metodun *necə* edəcəyini göstərir (gövdə, yəni kod).

```
// Bu sadəcə deklarasiyadır (abstract metod) — implementasiya yoxdur
abstract class Shape
{
    public abstract double GetArea();
}

// Burada isə implementasiya var
class Circle : Shape
{
    private double radius;
    public Circle(double radius) => this.radius = radius;

    // Implementasiya → necə edəcəyini yazırıq
    public override double GetArea()
    {
        return Math.PI * radius * radius;
    }
}
```

Qısa desək:
•**Deklarasiya** → *"Bu işi görəcəyik"*
•**Implementasiya** → *"Bu işi belə edirik"*

Xüsusiyyətləri:

•**Implementasiya məcburidir**
Əgər class interface-i implement edirsə, bütün metodlarını override etməlidir.
•**State saxlamaz**
Field ola bilməz, amma property-lər ola bilər.
•**Çoxsaylı implementasiya mümkündür**
Bir class istənilən sayda interface implement edə bilər.
•**C# 8+ versiyasında default metod implementasiyası** əlavə olundu:

```
interface IShape
{
    double GetArea();
    void Display() => Console.WriteLine("This is a shape.");
}
```

```
interface IShape
{
    double GetArea();
}

class Rectangle : IShape
{
    private double width, height;
    public Rectangle(double w, double h) { width = w; height = h; }
    public double GetArea() => width * height;
}
```

Xüsusiyyət	Abstract Class	Interface
Obyekt yaradıla bilər?	✕	✕
Field (daxili dəyişən) saxlaya bilər?	✓	✕
Tam metod implementasiyası ola bilər?	✓	✓ *(C# 8+)*
Constructor ola bilər?	✓	✕
Çoxlu miras alma	✕ *(yalnız bir abstract class)*	✓ *(bir neçə interface)*
Performans fərqi	Bir az daha sürətli (çünki VTable birbaşa bağlanır)	Bəzən zəifcə yavaşdır (çoxlu implementasiya səbəbi ilə)
İstifadə məqsədi	Miras əlaqəsi, ortaq kod + ortaq kontrakt	Sadəcə kontrakt, çoxlu implementasiya imkanları

VTable nədir?

•**VTable** (Virtual Table — virtual cədvəl) obyekt yönümlü dillərdə (C#, C++, Java) **virtual metodların çağırılması üçün istifadə olunan daxili bir cədvəldir**.
•Hər class üçün, əgər virtual və ya override metod varsa, **CLR (Common Language Runtime)** həmin metodların ünvanlarını saxlayan bir cədvəl yaradır.
•Obyekt üzərindən metod çağıranda CLR həmin cədvəldən metodu tapıb çağırır.

Virtual açar sözü nədir?

- C#-da **virtual** — bir metodun və ya property-nin **miras alan class tərəfindən yenidən yazıla (override edilə) biləcəyini** bildirən açar sözdür.

- Normalda class-da yazdığın metod **sabit**dir — onu miras alan class dəyişdirə bilməz.
  Əgər metodu **virtual** olaraq işarələsən, o zaman **override** ilə onu dəyişmək olar.

```
using System;

class Animal
{
    public virtual void Speak()
    {
        Console.WriteLine("Animal sound");
    }
}

class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Hav Hav");
    }
}

class Program
{
    static void Main()
    {
        Animal animal = new Animal();
        animal.Speak(); // Animal sound

        Animal dog = new Dog();
        dog.Speak(); // Hav Hav (override edildi)
    }
}
```

Encapsulation nədir?

- Proqramlaşdırmada kapsullamağa datanı bir qoruma qalxanına almaq üçün ehtiyac duyuruq.Yəni kənardan birbaşa dataya çatmaq yerinə onu bir kapsulun içinə alıb müəyyən şərtlər qoyuruq və o dataya çatmaq istəyənə də, sanki “Burda bu kapsulun qaydaları keçir!” deyərək məhdudiyyət qoymuş oluruq.

- Proqramlaşdırmada kapsullamağa datanı bir qoruma qalxanına almaq üçün ehtiyac duyuruq.Yəni kənardan birbaşa dataya çatmaq yerinə onu bir kapsulun içinə alıb müəyyən şərtlər qoyuruq və o dataya çatmaq istəyənə də, sanki “Burda bu kapsulun qaydaları keçir!” deyərək məhdudiyyət qoymuş oluruq.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace OOP
{
public class Encapsulation
{

public string name;

}
}
```

#Məsələn belə bir situasiya ürəkaçandırmı?Sırf **string** tipində verilib deyə rəqəm təyin etmək heç də developerlərin sevəcəyi bir şey deyil.Bunun üçün yardımımıza **kapsullamaq** yetişir :

```
using System;

namespace OOP
{
class Program
{
static void Main(string[] args)
{
Encapsulation encapsulation = new
Encapsulation();
encapsulation.name = "54";
Console.WriteLine(encapsulation.n
ame);
}
}
}
using System;
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace OOP
{
public class Encapsulation
{
//1-ci addım : Dəyişənin əlçatanlığını "private" edirik
private string name;
//2-ci addım : Dəyişənə dəyər ötürmək,dəyərini almaq üçün "public" metodlar təyin
edirik və içində istədiyimiz məhdudiyyətləri qoyuruq
public string getName()
{
return this.name;
}
public void setName(string name)
{//Set metodunda belə bir məhdudiyyət qoyduq ki,əgər ötürülən dəyişən içində hərfdən
başqa bir şey varsa,ötürülmə baş verməsin
bool isThereExceptLetter = false;
foreach(char character in name)
{
if (!(Char.IsLetter(character)))
{
isThereExceptLetter = true;
}
}
//Və əgər nəticə "true" olarsa,if içindəki əməliyyat baş verəcək
if (isThereExceptLetter)
{
Console.WriteLine("Yalnız hərflərə icazə verilir");
}
}

}
}
```

```
using System;

namespace OOP
{
class Program
{
static void Main(string[] args)
{
Encapsulation encapsulation = new Encapsulation();
encapsulation.setName("54");
}
}
}
```

Yuxarıdakı misalda **set** prosesinə bir məhdudiyyət qoyduq,get üçün də məhdudiyyət qoymaq mümkündür ki, “hər yoldan keçən” dəyəri əldə edə bilməsin.

Niyə lazımdır?

•**Məlumatı qorumaq** → Class-ın daxilindəki dəyişənlərə istənilən yerdən müdaxilə edilməsin.
•**Nəzarət mexanizmi yaratmaq** → Hər dəyişiklik müəyyən qaydalara uyğun olsun.
•**Sadə interfeys təmin etmək** → İstifadəçi class-ın daxili işini bilmədən istifadə edə bilsin.
•**Kodun dəstəklənməsini artırmaq** → Gələcəkdə daxili implementasiya dəyişsə belə, xaricdən istifadə edənlərə təsir etmir.

Nəzarət mexanizmi yaratmaq

- Enkapsulyasiya sayəsində hər bir dəyişiklik **qaydalara uyğun aparılır**.
- Biz dəyişkənə verilən dəyəri yoxlaya, şərt qoya və lazımi vaxtda error ata bilərik.

```
public class Person
{
    private int age;

    public int Age
    {
        get { return age; }
        set
        {
            if (value >= 0 && value <= 120)
                age = value;
            else
                throw new ArgumentException("Yaş düzgün deyil!");
        }
    }
}
```

Sadə interfeys təmin etmək

- İstifadəçi class-ın **daxili işini bilmədən** rahat istifadə etməlidir.
- Class-ın içində minlərlə hesablamalar ola bilər, amma istifadəçi sadəcə **asand metodları** çağırır.
- İstifadəçi sadəcə Start() və Drive() metodlarını çağırır,daxildə mühərrikin necə işə düşdüyünü bilmir.

```
public class Car
{
    private bool engineStarted;

    public void Start()
    {
        // daxildə mühərriki işə salmaq üçün 10 addım ola bilər
        engineStarted = true;
        Console.WriteLine("Mühərrik işə düşdü!");
    }

    public void Drive()
    {
        if (engineStarted)
            Console.WriteLine("Maşın gedir...");
        else
            Console.WriteLine("əvvəl mühərriki işə salın!");
    }
}
```

Kodun dəstəklənməsini artırmaq

•Əgər class-ın daxili implementasiyası dəyişsə belə, istifadəçilər eyni interfeysdən istifadə etdiyinə görə onların kodu **pozulmur**.
•Bu, **“loose coupling”** (zəif asılılıq) prinsipinə xidmət edir.
•Gələcəkdə temperaturu Kelvin ilə saxlamağa qərar versən,daxili kod dəyişməyəvək,amma istifadəçilər Celsius və Fahrenheit propertylərindən eyni qaydada istifadə etməyə davam edəcəklər.

```
public class Temperature
{
    private double celsius;

    public double Celsius
    {
        get { return celsius; }
        set { celsius = value; }
    }

    public double Fahrenheit
    {
        get { return celsius * 9 / 5 + 32; }
    }
}
```

Inheritance nədir?

- Inheritance OOP-nin ən önəmli özəlliyidir.

- *Yaradılan obyektlər fərqli obyektlərə özəlliklərini ötürə bilməkdə və beləliklə iyerarxik tənzimləmə edilə bilməkdədir.*

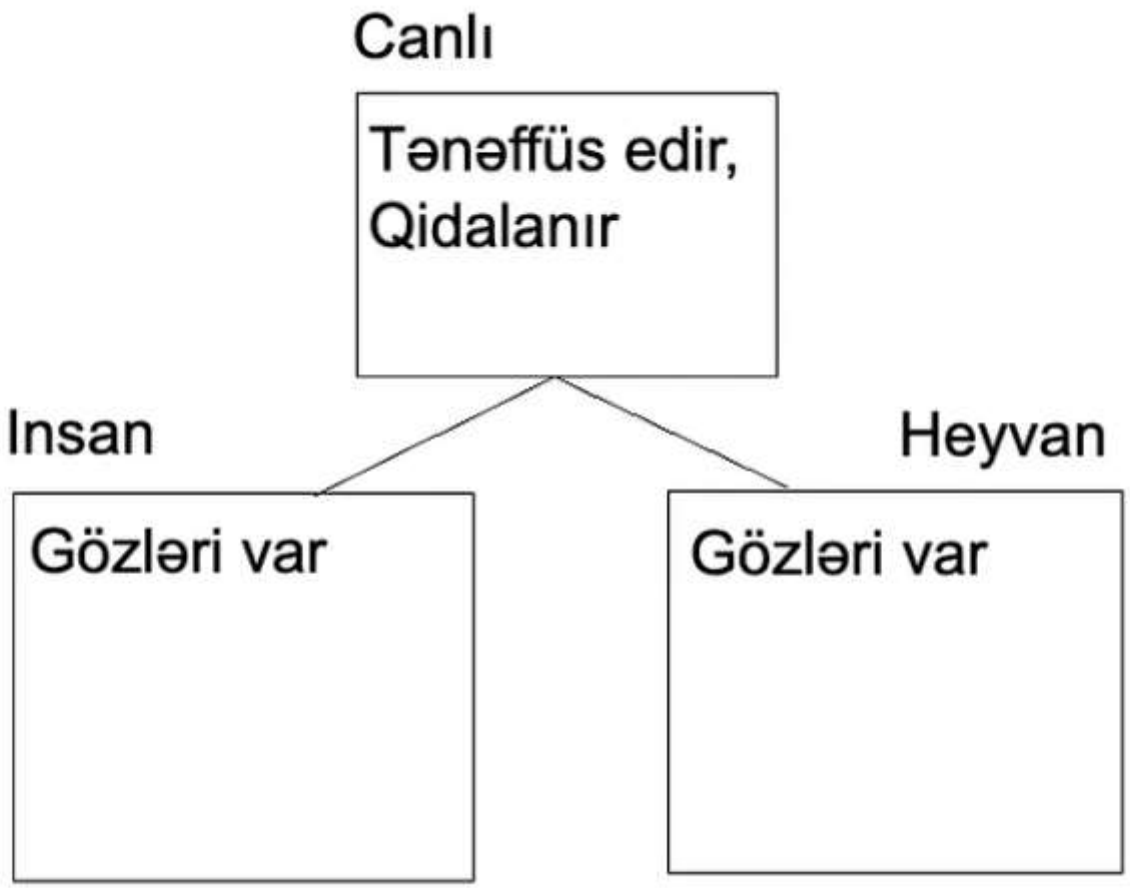
- Yəni,biz OOP-də obyektlərimizin bəzi özəlliklərini(field,metod,property…) başqa obyektlərə Miras olaraq ötürə bilirik.Bu ötürmə işinə biz İnheritance(Miras) deyirik.

- Eyni qrupdan olan obyektlərin oxşar özəlliklərini təkrar-təkrar hər birində yaratmaqdansa,bir üst class-da yaradılmasını və hər bir class-ın bu özəllikləri üst class-dan miras olaraq almasını təmin edir.

- Gəlin bir misal verim:Misal üçün qadın və kişi hər ikisidə eyni qrupdan gəlir.Yəni,hər ikisi də insandır.Buna görə də,hər ikisinin də ortaq özəllikləri var.
- Kodlama zamanı bu ortaq özəllikləri tək-tək hər iki class-da ayrı-ayrı yaratmaqdansa,üst class-da bu ortaq özəllikləri yazarıq və hər iki class-da bu class-dan miras ala bilər.
- Beləliklə,həm kod maliyyəti düşməkdə,həm də arxitektural dizayn cəhətdən sərfəli olmaqdadır.
- Nəticə etibarilə deyə bilərik ki:Eyni fitrətə sahib obyektləri təmsil edən və içərisində birə-bir eyni olan memberları(field-ları,property-ləri,özəllikləri və s.) hər biri üçün təkrar-təkrar yaratmaqdansa,sadəcə bir üst class yaradıb və bu özəllikləri burada yaradıb daha sonra həmən class-lara miras olaraq verə bilərik.

Bəs hər şeydə OOP nin inheritance prinsipindən istifadə etməliyikmi?

- Əslində belə baxsaq insanlar heyvanlar və bitkiləri də canlılar sinfinə aid etmək olar.Çünki hər 3 ü də qidalanır tənəffüs edir və s yəni bir neçə ortaq xüsusiyyətləri var.Yəni bu o demək deyil ki,hər dəfə bu prinsipi istifadə etməyə çalışaq.Lazım olduqca istifadə etmək əlverişlidir.


Canlı
Tənəffüs edir,
Qidalanır
Insan
Gözləri var
Heyvan
Gözləri var

```
namespace İnheritance
{
    2 references
    public class Canlı
    {
        3 references
        public string Ad { get; set; }
        2 references
        public int Yas { get; set; }
        2 references
        public void NəfəsAl()
        {
            Console.WriteLine($"{Ad} nəfəs alır.");
        }
    }
}
```

```
namespace İnheritance
{
    2 references
    public class İnsan : Canlı
    {
    }
}
```

```
namespace İnheritance
{
    2 references
    public class Heyvan : Canlı
    {
    }
}
```

C#-da hansı strukturlar miras ala bilirlər?

- *C# proqramlaşdırma dilində İnheritance class-lara xas bir xüsusiyyətdir.*

- Yəni bir class sadəcə və sadəcə başqa bir class-dan miras ala bilməkdədir.
- Burada sual verə bilərsiniz ki:Abstract class,Interface-lərdən miras ala bilmirikmi?

Bəs abstract class və interface miras vermir?inheritance prinsipi ödənmir?

- Abstract class də bir class –dır,sadəcə tam implement olunmamışdır.
- Ona görə abstract class-dan miras almaq inheritance sayılır.
- Sadəcə fərq odur ki,içindəki abstract metodları override etmək lazımdır.

```
abstract class Shape
{
    public abstract double GetArea();
}

class Circle : Shape
{
    public double Radius { get; set; }
    public Circle(double r) => Radius = r;

    public override double GetArea() => Math.PI * Radius * Radius;
}
```

Interface implementation

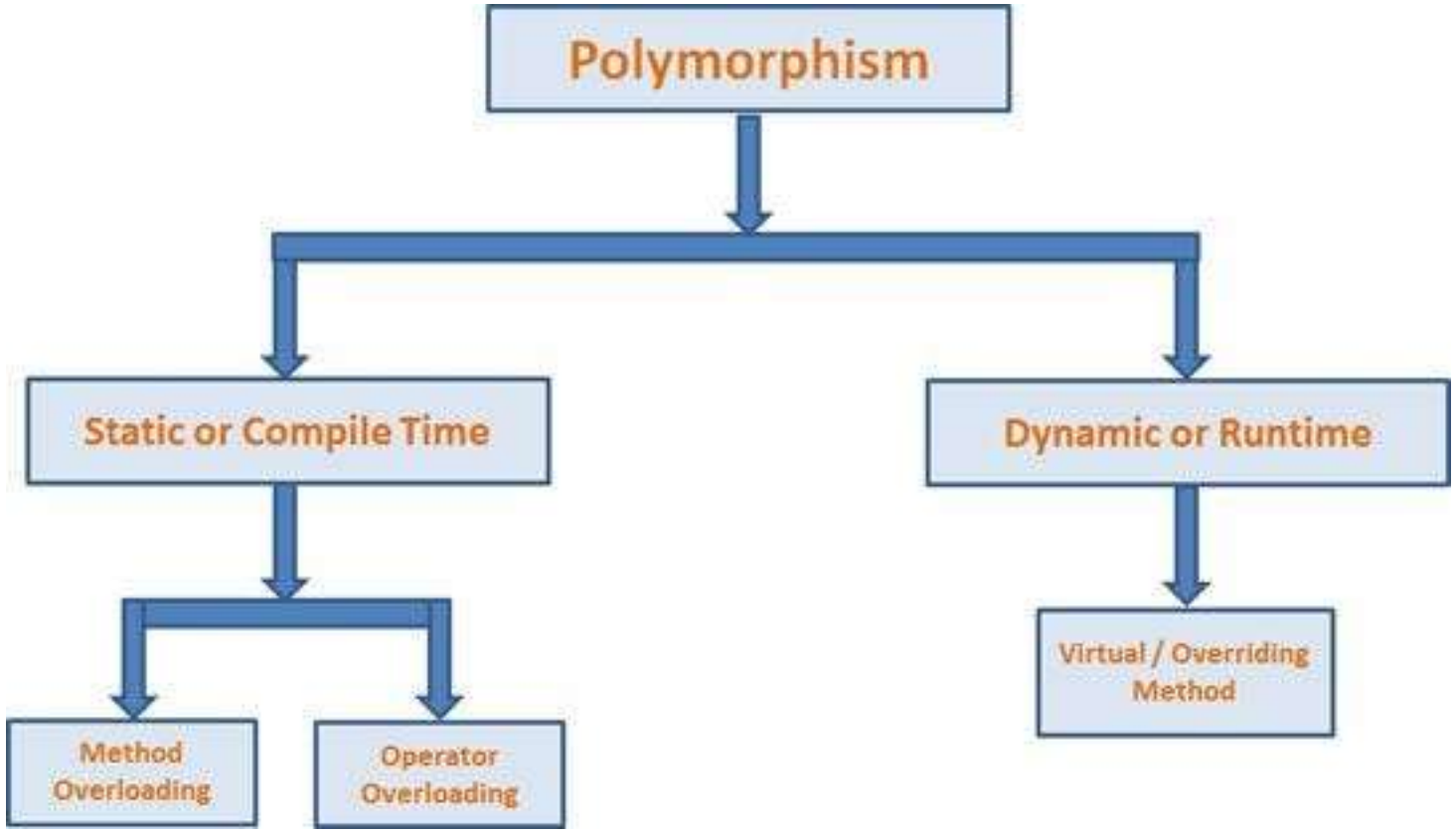
- İnterface – dən istifadə etdikdə biz inheritance yox implementation edirik.
- Amma texniki olaraq bu da polymorphism prinsipini reallaşdırır.

```
interface IMovable
{
    void Move();
}

class Car : IMovable
{
    public void Move() => Console.WriteLine("Car is moving...");
}
```

Polimorfizm

- **Polymorphism-**yunanca bir sözdür, “bir ad çox formadır” mənasını verir. Polimorfizm bir classda eyni adlı methoddan çoxsaylı tətbiqetmə imkanı verir. Polimorfizm-2 yerə bölünür. Static və Dinamik.


Polymorphism
Static or Compile Time
Dynamic or Runtime
Method Overloading
Operator Overloading
Virtual / Overriding Method

Compile-time (Kompilyasiya zamanı)

•**Nə vaxt olur?**
Kod **kompilyator** tərəfindən çevriləndə, yəni mənbə kodu (source code) maşın koduna və ya ara intermediate koda (məs. .NET-də IL, Java-da bytecode) çevriləndə.
•**Xüsusiyyətlər:**
•Sintaksis səhvləri (syntax errors) burada aşkarlanır.
•Tip uyğunluğu (type checking), dəyişənlərin mövcudluğu, metod imzalarının düzgünlüyü kimi məsələlər burada yoxlanır.
•Performansa təsiri yoxdur, çünki hələ proqram işə düşməyib.

Run-time (İcra zamanı)

•**Nə vaxt olur?**
Proqram **işlədikdə** (istifadəçi açanda, kod icra olunanda).
•**Xüsusiyyətlər:**
•Compile zamanı hər şey düzgün görünə bilər, amma icra zamanı sistem resursu, fayl yolu, ədədi sıfıra bölmək kimi məsələlərdə səhvlər yarana bilər.
•Bu səhvlər yalnız proqram işləyərkən üzə çıxır.

Statik Polimorfizm

- Static Polymorphism- eyni adlı methodun parametr sayına və type(int,string..)görə fərqlənməsidir.

- Statik polimorfizm (bəzən **compile-time polymorphism** də deyirlər) OOP-da bir obyektin eyni adda, amma fərqli imzalarla (parametrlərlə) bir neçə metodu icra etməsi imkanını bildirir. Bu polimorfizm növü **compile zamanı** həll olunur (metod çağırışı hansı imzaya düşəcək, kompilyator əvvəlcədən bilir).

```
public class Calculator
{
    // Metod overload (Statik Polimorfizm)
    public int Add(int a, int b)
    {
        return a + b;
    }

    public double Add(double a, double b)
    {
        return a + b;
    }

    public int Add(int a, int b, int c)
    {
        return a + b + c;
    }
}

class Program
{
    static void Main()
    {
        Calculator calc = new Calculator();

        Console.WriteLine(calc.Add(2, 3));          // int versiyası
        Console.WriteLine(calc.Add(2.5, 3.5));      // double versiyası
        Console.WriteLine(calc.Add(1, 2, 3));       // 3 parametrli versiyası
    }
}
```

Operator overload

- Bir sinif və ya structur üçün C# da – mövcud olan operatorların (+,-,*,/,==) özəl davranışını təyin etməyə çalışırıq.

- Yəni normalda + yalnız ədədləri toplayır ,amma sən obyekt tipinə + operatorunu yükləməklə onu fərqli cür işlədə bilərsən.

```
public struct Point
{
    public int X, Y;

    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }

    // + operatorunu yüklayirik (iki Point toplamaq)
    public static Point operator +(Point a, Point b)
    {
        return new Point(a.X + b.X, a.Y + b.Y);
    }

    // == operatorunu yüklayirik (iki Point bərabərdir?)
    public static bool operator ==(Point a, Point b)
    {
        return a.X == b.X && a.Y == b.Y;
    }

    public static bool operator !=(Point a, Point b)
    {
        return !(a == b);
    }
}

class Program
{
    static void Main()
    {
        Point p1 = new Point(1, 2);
        Point p2 = new Point(3, 4);

        Point p3 = p1 + p2;       // + operatoru özəl işləyir
        Console.WriteLine($"{p3.X}, {p3.Y}"); // 4, 6

        Console.WriteLine(p1 == p2); // False
    }
}
```

Dinamik Polimorfizm(Runtime)

- Törəmədə istifadə etditimiz virtual-override misalından aydın görə bilərsiz.

- Bir baz (base) class-dan törəyən siniflər eyni adlı metodu **fərqli implementasiya** ilə yaza bilirlər. Proqram çalışanda (runtime) obyektin real tipinə baxaraq uyğun metod çağırılır.

- Base klas da metod virtual və ya abstrakt olmalıdır.
- Derived klasda metod override ilə yenidən yazılır.
- Metod çağırışı – dəyişənin tipi base class ola daçreal obyektin tipi derived class olduğu üçün həmin obyektin öz overridə edilmiş metodu çağrılır.

Copy code

```
public class Animal
{
    public virtual void Speak()
    {
        Console.WriteLine("Animal makes a sound");
    }
}

public class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Dog barks");
    }
}

public class Cat : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Cat meows");
    }
}

class Program
{
    static void Main()
    {
        Animal a1 = new Dog();  // baxmayaraq ki, dəyişənin tipi Animal-dır
        Animal a2 = new Cat();

        a1.Speak(); // "Dog barks" -> runtime-da Dog obyektinin Speak metodu çağırıldı
        a2.Speak(); // "Cat meows" -> runtime-da Cat obyektinin Speak metodu çağırıldı
    }
}
```